

1. VQA: asking questions about the picture

Example: “Is there a river visible in this image?”

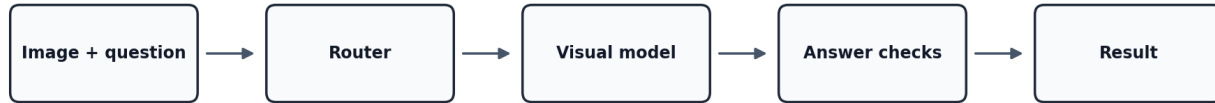


Image + question → router → visual model → answer checks → result

1. The user uploads a satellite image.

Our local Python server receives it through `/api/images`. It checks whether the file is supported and prepares the picture for the model.

2. The user types a question and clicks Analyze.

The question goes to `/api/analyze`. These APIs are doors inside our own application—not calls to ChatGPT or an external AI service.

3. Our router decides who should answer.

It follows written rules. A supported question about visible features goes to VQA. An unsupported request gets an explanation.

4. The visual model reads the image and question together.

We use the existing **AdaptLLM remote-sensing Qwen2-VL model**, running locally. It turns the picture into information it can process, connects that information with the question, and generates an answer.

If the user selects an experimental mode, our **LoRA adapter** changes how the model responds. Think of the adapter as a small set of learned notes attached to the existing model.

5. Our answer-handling code checks the generated text.

It removes certain speculative explanations and withholds ambiguous answers. It cannot prove that every remaining observation is correct.

6. We show the answer and save the run details.

We retain the original model output, the displayed answer and the model used.

What calculations happen?

The model performs learned pattern recognition. It is not calculating river presence using a fixed scientific formula.

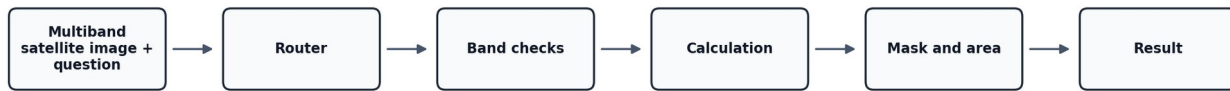
Say this to judges:

“We take the image and question, check that the question is supported, and send both to our local visual model. The model interprets the picture. Our application then applies answer checks and displays the result while preserving the original output.”

For baseball counting: the model counts the original image and a horizontally mirrored version. If both return the same readable number, we display the estimate. If they disagree, we withhold it. Agreement helps catch some errors, but it does not prove correctness.

2. NDVI: calculating from satellite measurements

Example: “Calculate NDVI coverage at the selected threshold.”



Multiband satellite image + question → router → band checks → calculation → mask and area → result

1. The user uploads a suitable satellite GeoTIFF.

This file contains separate measurements called **bands**. We need labelled, calibrated **red** and **near-infrared**, or **NIR**, bands.

An ordinary PNG only contains a visible picture, so it cannot supply this measurement.

2. The question reaches the same analysis API.

Our router recognises an NDVI request and sends it to the numerical tool.

The language model does not calculate the NDVI answer.

3. Our geospatial code checks the data.

It checks the band information and excludes unusable pixels—for example, invalid measurements and pixels rejected by the available quality mask. Calibration means the stored numbers must represent the correct light measurements.

4. For each usable pixel, we calculate:

$$\text{NDVI} = (\text{NIR} - \text{Red}) \div (\text{NIR} + \text{Red})$$

For example, if NIR is **0.6** and red is **0.2**:

$$\text{NDVI} = 0.4 \div 0.8 = 0.5$$

Vegetation often reflects more NIR than red light. NDVI describes that relationship; it is not automatically a plant-health diagnosis.

5. We apply the user’s threshold.

If the threshold is **0.50**, we mark usable pixels whose NDVI is at least **0.50**.

6. We count the marked pixels and calculate coverage.

$$\text{Coverage} = \text{marked pixels} \div \text{usable pixels} \times 100$$

When the image has suitable projected map coordinates, we also calculate:

$$\text{Selected area} = \text{marked pixels} \times \text{area of one pixel}$$

In our Dehradun sample, each pixel covers **10 × 10 metres = 100 square metres**.

$$38,761 \text{ marked pixels} \times 100 = 3,876,100 \text{ square metres} = 387.61 \text{ hectares.}$$

7. We show the numbers and their visual evidence.

The application produces the overlay, threshold mask and downloadable NDVI raster. These come from the calculation—not from the model drawing a convincing-looking picture.

Say this to judges:

“We take the satellite’s red and near-infrared measurements and calculate NDVI for each usable pixel. We apply the selected threshold, count the matching pixels and calculate their coverage and mapped area. We then show exactly which pixels contributed to the result.”

3. Grounding: a future flow for locating objects

Example: “Show me the buildings in this image.”

Grounding means **connecting a phrase to an actual place in the picture**, usually with boxes or outlines. **We do not currently have this working specialist.**

The proposed flow would be:

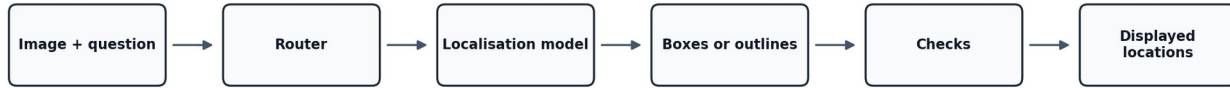


Image + question → router → localisation model → boxes or outlines → checks → displayed locations

1. The user uploads an image and asks to locate something.

2. We add a supported grounding route.

Initially, we would support a small set of object types that we have tested.

3. We add a model that can locate objects.

Our current text-answering path does not provide validated object locations. We would need to select and integrate a suitable **detector or grounding model**.

4. That specialist returns positions in the image.

For buildings, it might return a rectangle around each detected building. If we later need precise outlines, we could add a **segmentation model** that selects the object’s pixels.

5. Our application draws those positions over the original image.

If the input has reliable geographic information, we could then convert image positions into map coordinates. The geographic conversion does not discover the buildings; the localisation model must do that first.

6. Before enabling it, we verify the locations.

We would use separate images where people have marked the real objects, then check:

- Did it find the correct objects?
- Did it miss objects or invent extra ones?
- Do its boxes or outlines fit the objects?
- Does it work on new locations and at usable speed?

7. Only after those checks would we expose the tested capability.

We would keep unsupported categories and unreliable cases outside the supported scope.

Say this to judges:

“For grounding, we would add a specialist that returns object positions rather than just words. We would compare its boxes against human-marked examples on separate images before enabling it. Our current NDVI mask marks a numerical threshold; it does not locate named objects such as buildings.”